

TradeVault

Audit Architecture IA & Automatisations

AI Systems Architect - Pre-Funding

Juillet 2026

Analyse de l'architecture IA, agents, prompts, outils, memoire,
routage, couts, automatisations, event bus et personnalisation

Ne pas diffuser en externe

Methodologie

Analyse de l'ensemble de l'architecture IA et des automatisations de TradeVault en tant qu'AI Systems Architect. L'analyse couvre Jarvis, les agents, les prompts, le tool calling, la memoire, la personnalisation, le contexte envoye aux modeles, les couts, le routage des modeles, les Server Functions IA, les notifications intelligentes, l'onboarding, les automatisations, les emails, les push notifications, les taches planifiees, les workflows, l'Event Bus et la communication entre modules.

Les doublons avec les audits precedents (Architecture, Performance, Securite, Mobile) sont exclus. Chaque probleme identifie identifie les fichiers concernes, l'impact, la solution, la priorite (P0/P1/P2) et l'estimation du temps de correction.

1. Vue d'Ensemble de la Plateforme IA

Architecture actuelle

TradeVault dispose d'une plateforme IA remarquablement architecturée avec des abstractions propres : provider-agnostic (OpenAI, Anthropic, Gemini interchangeables par variable d'environnement), plug-in registries (agents, tools, jobs, formatters), Context Builder capable, prompt builder typ, et event-driven automation engine. Le code est propre, testable et bien séparé des responsabilités. Cependant, une proportion significative du code est à l'état de contrat/fondation sans implémentation réelle.

Modules IA : 11 modules dans `src/modules/ai/`, 1 module `src/modules/ai-provider/`, 6 fichiers backend `src/backend/` (*.functions.ts), 6 fichiers UI. Total ~2000 lignes de code. Environ 40% est du scaffolding (contrats, types, registres vides).

Forces identifiées

- Provider-agnostique: changer de modèle = une variable d'env, zero refactor.
- Fallback déterministe: `fallback-coach.ts` produit des réponses zero-cout sans IA.
- Contexte construit côté client: les données sont déjà dans le cache React Query.
- Caps multi-couches: Zod serveur + ContextBuilder côté client + maxTokens.
- Pipeline d'automation: AutomationEngine avec steps pluggables et error isolation.
- Bus d'événements type avec 18 événements définis, error-isolation par listener.
- Mémoire persistée (`ai_memory`) avec RLS owner-only.
- Telemetry hook (`UsageEvent`) prêt à être connecté.
- Coach déterministe de fallback: zero cout, aucune dépendance réseau.

2. Problemes Critiques (P0)

P0 Tool Calling: Zero Outils Metier Implementes

Page: Plateforme IA - capacite d'action des agents | modules/ai/tools/types.ts:11, modules/ai/agents/catalog.ts:22-62

Impact: Le systeme de tool calling a des contrats complets (ToolDefinition, registerTool, toolManifest, runtime d'execution avec boucle de tool calling dans provider-service.ts:121-142), mais registerTool() n'a JAMAIS ete appele. Les 5 agents declarent 14 utilisations d'outils dans le catalogue (get_stats, get_trades, search_memory, get_rules, get_goals, compute_quant_stats, get_discipline_events, assess_trade_risk). toolManifest() retourne un tableau vide pour chaque agent. Aucun outil n'existe en runtime.

Solution: Implementer les 6 outils fondamentaux: get_stats (lecture stats precalculees), get_trades (recherche de trades), get_rules (regles du trader), get_goals (objectifs), search_memory (recherche memoire persistee), get_discipline_events (evenements de discipline). Chaque outil doit etre enregistre via registerTool() et testable unitairement.

Correction: 3-4 jours

P0 Zero Proactivite IA - Coach Explicitement Reactif

Page: Architecture IA - capacite d'initiative | modules/ai/agents/coach.agent.ts:11, modules/events/types.ts:30-48

Impact: Le coach agent V1 declare a la ligne 11 : 'NO proactivity (no jobs, no notifications)'. Les evenements definis pour la proactivite IA (DailyBriefReady, WeeklyReviewReady, NewPatternDetected, DailyReset) ne sont JAMAIS emis. Les 6 evenements AI OS (AgentRunStarted, AgentRunCompleted, AiJobEnqueued, AiJobCompleted, DocumentIndexed, AiNotificationRequested) sont definis mais jamais utilises. L'IA ne peut jamais prendre d'initiative - pas de brief quotidien pousse, pas d'alerte de pattern, pas de coaching proactif.

Solution: Wiring chron -> generation IA -> emission d'evenement -> notification. Pour le daily brief: cron quotidien -> appelle aiGenerateDailyBrief() -> emet DailyBriefReady -> NotificationEngine envoie push/email. Meme pattern pour weekly review et pattern detection.

Correction: 2-3 jours

P0 Aucun Suivi de Cout ou Telemetrie IA en Runtime

Page: Plateforme IA - observabilite et couts | modules/ai/telemetry.ts:9, modules/ai/provider-service.ts:28-35

Impact: Le contrat de telemetrie (telemetry.ts) est anote 'FOUNDATION ONLY: no writer yet' (ligne 9). Le hook UsageEvent dans provider-service.ts est declenche a chaque appel mais personne n'ecoute - aucun recorder n'est branche. Aucun cout par utilisateur, aucun suivi de tokens, pas d'arbitrage de modele base sur le cout. Impossible de detecter les abus ou d'optimiser la selection de modele par cas d'usage.

Solution: Implementer un TelemetryRecorder qui ecrit dans la table ai_agent_runs (migration existe). Brancher le callback onUsage dans les server functions (coach.functions.ts, ai.functions.ts). Ajouter un tableau de bord des couts par utilisateur pour les administrateurs.

Correction: 2-3 jours

P0 Fire-and-Forget dans App.tsx Perd Silencieusement les Resultats d'Automation

Page: Fiabilite du pipeline d'automation | src/app/App.tsx:196-208

Impact: Le pipeline d'automation est lance avec 'void (async () => { await AutomationEngine.tradeSaved(...); })();' - promesse non await. Si l'analyse de trade, la discipline ou les evenements echouent, l'erreur est silencieusement ignoree. Aucun feedback a l'utilisateur, aucune observabilite. Des echecs multiples dans une session se cumulent sans alerte.

Solution: Soit (a) capturer l'erreur et emettre un toast utilisateur + log serveur, soit (b) utiliser une file d'attente background (AiJob) pour les automations. Option (a) en 0.5 jour, option (b) en 2 jours mais plus robuste.

Correction: 0.5 jour (option a)

P0 50% des Evenements Domaine Jamais Emis - Evenements Morts

Page: Architecture evenementielle - completude | modules/events/types.ts:26-48, modules/events/bus.ts:37-40

Impact: 9 des 18 evenements definis ne sont jamais emis: GoalUpdated, GoalCompleted, DailyBriefReady, WeeklyReviewReady, NewPatternDetected, DailyReset, et les 6 evenements AI OS n'ont AUCUN emitter. De plus, le bus.ts:38-40 utilise fire-and-forget pour les handlers asynchrones - les erreurs sont loggees mais les promesses jamais attendues. L'architecture 'adding a feature = adding an event + listener' est brisee par le maillon manquant des emetteurs.

Solution: Pour chaque evenement mort: (1) creer l'emetteur dans le module concerne, (2) creer le handler/abonne dans le module consommateur, (3) faire en sorte que EventBus.emit retourne Promise<void> quand il y a des handlers async pour que les emetteurs puissent await.

Correction: 3-4 jours

P0 Tous les 500 Trades Envoyes a Chaque Appel IA - Gaspillage de Tokens

Page: Performance et cout des appels IA | modules/ai/context.ts:108-110, src/backend/ai.functions.ts:33

Impact: Le contexte envoie jusqu'a 500 trades (AIUserContext.trades cap a 500) meme quand la question ne necessite que des stats. Le bloc RECENT TRADES est le plus gros consommateur de tokens: $\sim 500 \text{ trades} \times \sim 200 \text{ caracteres} = \sim 100\,000 \text{ caracteres}$ par appel. Un daily brief de 200 mots paie pour 500 resumes de trades dont il n'a pas besoin.

Solution: Implementer une selection intelligente du contexte: par default n'envoyer que les 20 trades les plus recents pour le chat general. Pour les questions specifiques (analyse de pattern, recherche par symbole), envoyer un sous-ensemble pertinente. Utiliser l'intention detectee pour dimensionner le contexte.

Correction: 2 jours

P0 Memoire IA Croit sans Limite - Pas de Compactage ni Consolidation

Page: Qualite de la memoire long-terme | modules/ai/memory.ts:24-43, modules/ai/rag/types.ts (foundation)

Impact: Le systeme de memoire n'a aucun mecanisme d'oubli, de compactage ou de scoring d'importance. loadMemory() retourne les entrees brutes trieess par date, capees a 40-60 items. Jusqu'a 60 entrees $\times$ 2000 caracteres = 120KB de memoire par appel. Pas de memoire semantique/RAG (rag/types.ts: foundation only). Les faits anciens importants se noient dans le bruit des entrees recentes.

Solution: Implementer la consolidation de memoire: resumer les entrees anciennes en faits durables (background job hebdomadaire). Implementer l'importance scoring pour retenir les faits cruciaux. La RAG (recherche vectorielle) est le plan 12-18 mois, mais un scoring heuristique peut etre fait en 4-5 jours.

Correction: 4-5 jours

3. Problemes Haute Priorite (P1)

P1 1 Agent sur 5 Implemente (Coach Seulement)

Page: Plateforme IA - couverture fonctionnelle | modules/ai/agents/catalog.ts:14-65, modules/ai/router/types.ts:48-57

Impact: Le catalogue definit 5 agents (coach, performance-analyst, psychologist, risk-manager, pattern-finder) avec des personae, outils et formats de sortie distincts. Seul coach.agent.ts a une implementation run(). Les 4 autres sont du metadata pur. Le routeur (router.ts) mappe les intentions vers des agents qui n'existent pas - INTENT_AGENT mappe weekly_review -> performance-analyst, detect_patterns -> pattern-finder, etc. La plateforme IA est a 20% de son intelligence prevue.

Solution: Implementer les 4 agents restants. Chacun consomme les sorties des moteurs deterministes (stats, analysis, discipline) via un contexte specialise. L'agent psychologist merite d'etre prioritaire car il est differenciant pour la retention (coaching comportemental).

Correction: 5-7 jours par agent (total ~20-28 jours)

P1 Pas de Versioning de Prompts ni de Tests A/B

Page: Qualite et iterativebilite des prompts | src/backend/ai.functions.ts:54-55,89,106-107, modules/ai/agents/coach.agent.ts:60-120

Impact: Tous les prompts sont codes en dur dans le source (ai.functions.ts:54-55: COACH_IDENTITY en string template, coach.agent.ts:60-120: identity + anti-hallucination + format). Aucun template versionne, aucun controle de version, aucune infrastructure de test A/B. Modifier un prompt = editer le source + redeployer. Impossible de mesurer la qualite des prompts ou de revenir en arriere.

Solution: Extraire les prompts dans un store de templates versionne (JSON ou fichier dedie). Implementer un systeme de A/B testing avec evaluation: une question gold standard, deux variantes de prompt, comparaison des reponses. Ajouter une interface admin pour modifier les prompts sans redploiement.

Correction: 3-5 jours

P1 Routage de Modele: Un Seul Provider pour Tout

Page: Optimisation cout/qualite | modules/ai/router/router.ts:24, src/backend/ai.functions.ts:92

Impact: La variable AI_PROVIDER controle un seul provider pour tous les appels. Le routeur a une fente selectModel() mais personne ne l'appelle. Les requetes simples (chat, daily brief) et complexes (weekly review, pattern analysis) utilisent le meme modele. Soit on paie GPT-4 pour dire 'bonjour', soit on a une qualite mediocre sur les analyses complexes avec un modele cheap.

Solution: Implementer le routage par intention: modele cheap (Gemini Flash / GPT-4o-mini) pour chat/FAQ/daily brief, modele premium (Claude Sonnet/GPT-4o) pour weekly review/pattern analysis/risk assessment. Utiliser selectModel() du routeur.

Correction: 2-3 jours

P1 Dedup des Notifications Push: localStorage au Lieu du Serveur

Page: Fiabilite des notifications multi-appareils | modules/notifications/engine.ts:38-55

Impact: L'anti-spam push (alreadyPushedToday) stocke son etat dans localStorage (ligne 41-44). Chaque navigateur/appareil a son propre etat de dedup. Si un utilisateur a telephone + desktop, il recoit 2 pushes pour le meme evenement. La dedup est perdue si localStorage est efface.

Solution: Deplacer la dedup cote serveur: une table notification_dedup(user_id, dedup_key, date) avec upsert. Alternative: utiliser l'evenement NotificationCreated comme verrou - persister la notification d'abord, puis envoyer le push avec un check cote serveur.

Correction: 0.5-1 jour

P1 Pas de Gouvernance des Coûts IA par Utilisateur

Page: Maitrise des couts d'infrastructure | src/backend/require-pro.ts, modules/ai/provider-service.ts

Impact: Le rate limit est un plateau 60 req/h par utilisateur, independamment du modele ou du nombre de tokens. Pas de budget par utilisateur, pas d'alertes de depense, pas d'acces base sur le tier (free vs pro). Un utilisateur intensif utilisant GPT-4o pour 60 appels/h peut couter \$50+/jour sans aucun controle.

Solution: Implementer un budget tokens/jour par utilisateur/tier. Quand le budget est depasse: degager vers le modele le moins cher (fallback), ou bloquer avec un message. Ajouter des alertes quand un utilisateur depasse X% de son budget.

Correction: 2-3 jours

4. Problemes Moyens (P2)

P2 Plateforme IA ~40% de Scaffolding - 6 Modules sur 11 sont des Contrats Seulement

Page: Maintainabilite et dette technique | modules/ai/agents/types.ts, router/types.ts, tools/types.ts, rag/types.ts, jobs/types.ts, mcp/types.ts

Impact: Les modules agents/types.ts, router/types.ts, tools/types.ts, rag/types.ts, jobs/types.ts et mcp/types.ts sont a l'etat 'foundation only' - contrats/types sans implementation runtime. Les 6 evenements AI OS ne sont jamais emis. L'index.ts du module IA exporte des facades qui wrappent des registres vides. ~1500 lignes de scaffolding avec zero valeur runtime. Le cout de maintenance croit a chaque refactor.

Solution: Soit (a) supprimer les modules foundation-only jusqu'a ce que le financement permette leur implementation, soit (b) les deplacer dans un dossier /incubator clairement separe. Option (a) recommandee: reduire la surface de code sans valeur.

Correction: 1 jour

P2 Voix: Profil Statique Unique, Pas de Personalisation par Utilisateur

Page: Experience utilisateur vocale | modules/voice/profile.ts:14-24, modules/voice/localVoice.ts

Impact: JARVIS_VOICE est un profil singleton (voice/profile.ts) avec ElevenLabs voice ID PEuehAGHRfRnnYqUDPCo. La selection de voix navigateur dans localVoice.ts n'a pas de preferences utilisateur. Tous les utilisateurs entendent la meme voix, meme rythme, meme hauteur.

Solution: Stocker la preference vocale dans la table user_preferences. Permettre la selection de voix dans les parametres (Settings). Pour ElevenLabs: menu deroulant des voix disponibles. Pour la voix navigateur: slider de vitesse et hauteur.

Correction: 1-2 jours

P2 Donnees d'Onboarding Sous-Utilisees dans le Contexte IA

Page: Personalisation IA | src/app/utills/aiContext.ts:105-120

Impact: describeProfile() construit une ligne de profil mais manque plusieurs champs d'onboarding: brokers, experience detaillee, utilisation ICT, pain point principal ne sont pas toujours inclus dans le profil envoye a l'IA. Aucun parcours d'apprentissage personnalise base sur le niveau d'experience.

Solution: Elargir describeProfile() pour inclure tous les champs d'onboarding. Creer un objet de personalisation plus riche qui guide le coach sur le niveau de detail, le vocabulaire et les conseils adaptes au profil.

Correction: 0.5 jour

P2 Cron des Rapports Mensuels: Push Uniquement - Pas d'Email de Fallback

Page: Accessibilite des rapports | src/backend/monthly-reports.server.ts:223-253, modules/events/types.ts:30-31

Impact: monthly-reports.server.ts:223-253 envoie uniquement des notifications push. Les utilisateurs qui n'ont pas active les push ne savent jamais que leur rapport est pret. Aucun evenement DailyBriefReady ou WeeklyReviewReady n'est emis - ces evenements pourraient declencher d'autres canaux.

Solution: Ajouter un chemin email: envoyer un email de notification quand le rapport est genere (via Resend, comme les lifecycle emails). Emettre les evenements DailyBriefReady/WeeklyReviewReady pour permettre a d'autres abonnees de reagir.

Correction: 1-2 jours

P2 Pas de Framework d'Evaluation des Reponses IA

Page: Qualite et amelioration continue | Aucun fichier dedie - manque transversal

Impact: Aucun harness d'evaluation, aucun dataset golden, aucun test de regression sur les reponses IA. Pas de feedback utilisateur (pouce haut/bas sur les reponses). Pas de moyen de mesurer la qualite des reponses ou de detecter les regressions. Chaque changement de prompt est un saut dans l'inconnu.

Solution: Ajouter un widget de feedback sur les surfaces AI (pouce haut/bas + commentaire libre). Construire une suite d'evaluation avec des questions golden et des reponses attendues. Creer un tableau de bord qualite.

Correction: 3-5 jours

P2 Cycle de Vie des Objectifs Complettement Deconnecte

Page: Engagement et retention | modules/events/types.ts:26-27, implementation des goals

Impact: Les evenements GoalUpdated et GoalCompleted (events/types.ts:26-27) sont definis mais jamais emis. Aucune notification quand un objectif est atteint ou mis a jour. Aucune integration avec le coach IA pour reconnaitre les progres sur les objectifs. Les objectifs sont une checklist statique sans celebration ni coaching.

Solution: Emettre GoalUpdated/GoalCompleted depuis les operations CRUD des goals. Wiring: NotificationEngine ecoute ces evenements et envoie une notification avec le niveau de progression. Le coach IA peut référencer les progres dans ses reponses.

Correction: 1 jour

5. Event Bus et Automations

Etat actuel de l'Event Bus

L'Event Bus (modules/events/) est le systeme nerveux de TradeVault. Il definit 18 evenements de domaine. En excellent etat de conception: strongly-typed, error-isolated, singleton par runtime. Mais l'execution revele des lacunes critiques:

- 9 evenements sur 18 ne sont jamais emis (50% de taux d'utilisation)
- Les handlers async sont fire-and-forget (bus.ts:38-40) - leur promesse n'est jamais await
- Aucun subscriber pour TradeCreated, TradeUpdated, TradeDeleted, TradeAnalyzed
- GoalUpdated, GoalCompleted: definis mais sans emitter ni subscriber
- Tous les evenements AI OS (AgentRunStarted, AiJobEnqueued, etc.): definis, zero emission
- DailyReset: defini, zero emission

Notification Engine

Le Notification Engine est bien concu: adapters injectes (toast, push, persist), routing par canal, emission de NotificationCreated. Les points faibles:

- Dedup push en localStorage (per-appareil, pas per-utilisateur) - P1
- Pas de canal email implemente dans les adapters (bien que defini dans les types)
- Pas de fallback si tous les canaux echouent
- Aucune notification AI (daily_brief, weekly_review, pattern_detected) n'est jamais emise

Cron et Taches Planifiees

Deux crons Vercel: monthly-reports (1er du mois) et lifecycle-emails (quotidien). Goal reminders envoyes le lundi. Points faibles:

- Monthly reports: pas d'evenement emis, pas d'email de fallback, appel direct a Gemini (pas via le provider agnostique)
- Lifecycle emails: bien implementes avec dedup (email_log) et templates HTML. Mais pas de suivi des taux d'ouverture.
- Pas de cron pour daily brief ou weekly review AI - ces fonctionnalites existent dans ai.functions.ts mais ne sont jamais declenchees automatiquement

Pipeline d'Automation

AutomationEngine (modules/automation/engine.ts) a une architecture step-based propre (validate -> analyze -> discipline). Extensible via registerStep(). Mais:

- Lance en fire-and-forget depuis App.tsx (P0) - les echecs sont silencieux
- Pas de step de sauvegarde/rollback si une etape critique echoue
- Pas de monitoring du pipeline (combien de trades passes? combien d'echecs?)

5 Causes Racines de la Majorite des Problemes

1. Contrats avant implementation - l'architecture a ete dessinee pour le futur

Une grande partie des modules IA (tools, rag, jobs, mcp, router, agents) ont ete concus avec des contrats complets et des registres, mais sans implementation runtime. C'est une bonne pratique architecturale (planifier l'extension) mais elle cree 40% de code mort qui doit etre maintenu, documente et compris par chaque nouveau developpeur. Les 6 evenements AI OS sont definis mais non connectes. La tentation de 'preparer le futur' a cree de la dette.

Impact: ~40% du code IA est du scaffolding sans valeur runtime. Cout de maintenance transparent mais cumulatif. Les nouveaux developpeurs passent du temps a comprendre des systemes qui ne font rien.

Solution: Soit implementer les modules (priorite: agents > tools > router > jobs > rag > mcp) soit les deplacer dans un dossier /incubator avec un README clarifiant qu'ils sont des plans, pas du code live.

2. Pas de feedback loop utilisateur sur la qualite IA

Aucun mecanisme de retour utilisateur sur les reponses du coach (pouce haut/bas). Aucune evaluation systematique (golden dataset). Aucun suivi de qualite dans le temps. L'equipe ne peut pas savoir si une modification de prompt ameliore ou degrade l'experience. Le fallback deterministe est excellent mais cache les echecs du modele IA.

Impact: Impossible d'ameliorer systematiquement la qualite IA. Les regressions passent inaperceues. Les utilisateurs frustres partent sans donner de feedback.

Solution: Widget de feedback sur chaque reponse IA. Dataset golden de 50-100 questions avec reponses attendues. Harness d'evaluation automatise en CI.

3. La proactivite IA a ete differee - et ne viendra jamais sans architecture

Le coach V1 a explicitement exclu la proactivite (coach.agent.ts:11). Les evenements AI sont definis mais non connectes. Les fonctionnalites AI existent (daily brief, weekly review, pattern detection dans ai.functions.ts) mais ne sont jamais declenchees automatiquement. Aucun cron AI. La proactivite est le principal differentiateur d'un AI Trading OS vs un simple chatbot.

Impact: L'IA ne prend jamais d'initiative. L'utilisateur doit toujours demander. Pas de daily briefing du matin, pas d'alerte de pattern, pas de weekly review automatique.

Solution: Implementer le cron AI (daily brief chaque matin, pattern detection hebdomadaire). Wiring evenementiel: generation IA -> DailyBriefReady -> push/email.

4. Couts IA invisibles - pas de donnees pour arbitrer

Sans telemetrie, impossible de savoir combien coute chaque utilisateur, chaque intention, chaque modele. Sans cette donnee, impossible d'arbitrer entre cheap et premium (P1). Le rate limit a 60 req/h est un 'one-size-fits-all' qui sous-protege contre les abus et sur-protege les utilisateurs power.

Impact: Risque de facture AI inattendue. Pas de levier pour optimiser les couts. Pas de justification business pour le pricing Pro.

Solution: Telemetrie (P0) -> tableau de bord couts -> routage intelligent (P1) -> gouvernance par tier (P1).

5. L'Event Bus est concu comme un bus memoire, pas comme un bus de persistance

Le bus est 'per-runtime (non persiste)' par conception (docs/architecture.md:267). C'est un choix delibere, mais il signifie que les evenements sont perdus au rechargement de page. Les evenements critiques (TradeCreated, GoalCompleted) ne survivent pas au runtime. Cross-runtime delivery repose entierement sur les notifications persistees. C'est coherent mais limite: pas de replay d'evenements, pas de lecture differee, pas de event sourcing.

Impact: Les evenements sont volatils. Si un handler echoue et que la page est rechargee, l'evenement est perdu. Pas de possibilite de 'event-driven background jobs'.

Solution: Pour les cas d'usage critiques, ajouter une table domain_events avec emission persistee. Le bus memoire reste pour les notifications temps reel (toast, UI). Les jobs background utilisent la table persistee.

Score Global et Tableau Recapitulatif

Architecture & Design: 14/25 (56%)
Fonctionnalites & Completude: 5/20 (25%)
Performance & Couts: 5/15 (33%)
Fiabilite & Gestion d'Erreurs: 6/15 (40%)
Personalisation & UX: 5/10 (50%)
Event Bus & Automatisations: 4/10 (40%)
Tests & Observabilite: 1/5 (20%)
SCORE GLOBAL: 40/100 (40%)

Tableau Recapitulatif

Nombre total de problemes 18	
P0 - Critique	7
P1 - Haute priorite	5
P2 - Moyen	6
Temps d'implementation es ~30-45 jours/homme	

Top 10 Correctifs Prioritaires

- 1. Implementer les outils metier (get_stats, get_trades, search_memory...) (P0, 3-4j, debloque toute la capacite d'action des agents)
- 2. Telemetrie IA: brancher le UsageEvent hook + table ai_agent_runs (P0, 2-3j, visibilite couts + observabilite)
- 3. Proactivite IA: cron daily brief + weekly review + wiring evenementiel (P0, 2-3j, differentiateur AI OS vs chatbot)
- 4. Catch erreurs fire-and-forget App.tsx automation (P0, 0.5j, fiabilite pipeline)
- 5. Selection intelligente du contexte: 20 trades par default, contexte adaptatif (P0, 2j, reduction tokens 3-5x)
- 6. Event Bus: emettre les evenements morts + async handlers (P0, 3-4j, completude architecture evenementielle)
- 7. Routage de modele par intention (P1, 2-3j, optimisation cout/qualite)
- 8. Dedup push cote serveur (P1, 0.5-1j, fiabilite multi-appareil)
- 9. Budget tokens par utilisateur/tier (P1, 2-3j, gouvernance couts)

- 10. Widget feedback utilisateur + golden dataset evaluation (P2, 3-5j, amelioration continue qualite IA)